Segurança de Redes

Network Security

Mestrado em Cibersegurança (MCS)

Mestrado em Redes de Computadores (MRC)

Departamento de Informática, Universidade do Minho

Ivo Miguel Menezes Silva

1° Semestre (26/27)

Adapted from Prof. Paulo Carvalho's (Dept. Informática – UMinho) slides, which were originally based on:
J.F Kurose and K.W. Ross, "Computer Networking: A Top-Down Approach, 8th ed., Pearson, 2020. Teaching material.

Securing end-to-end connections
Transport Layer Security (TLS)

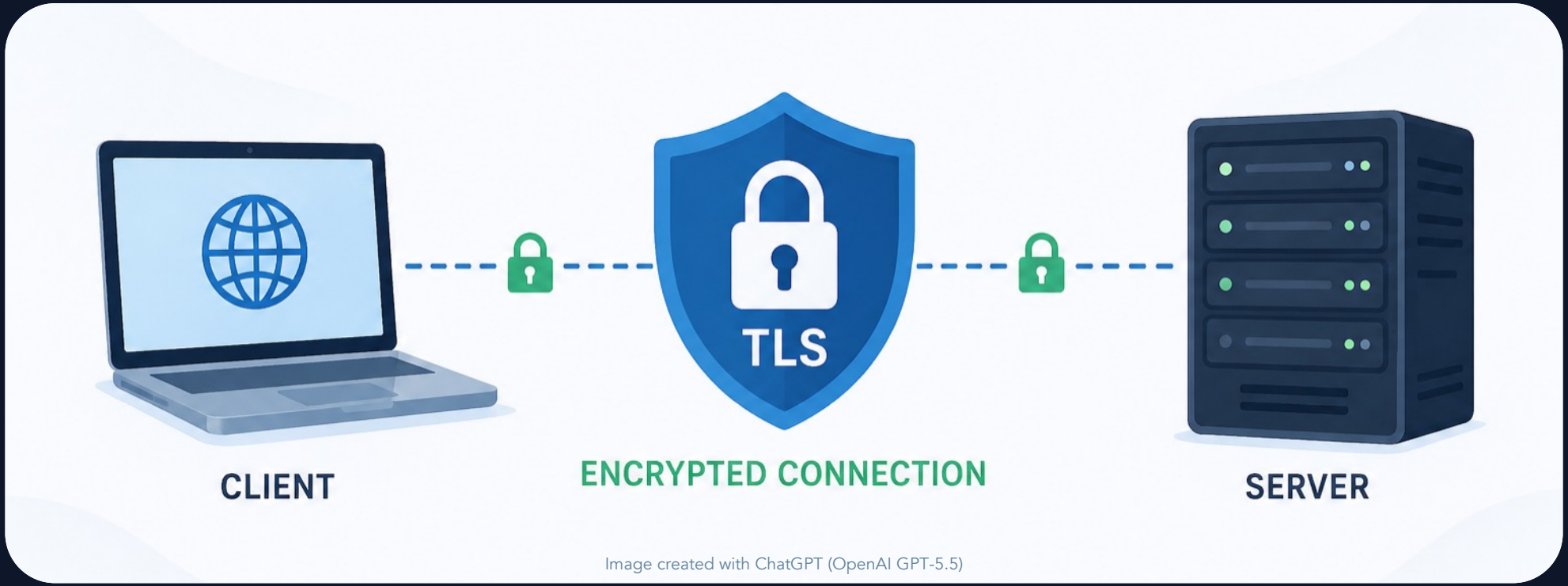
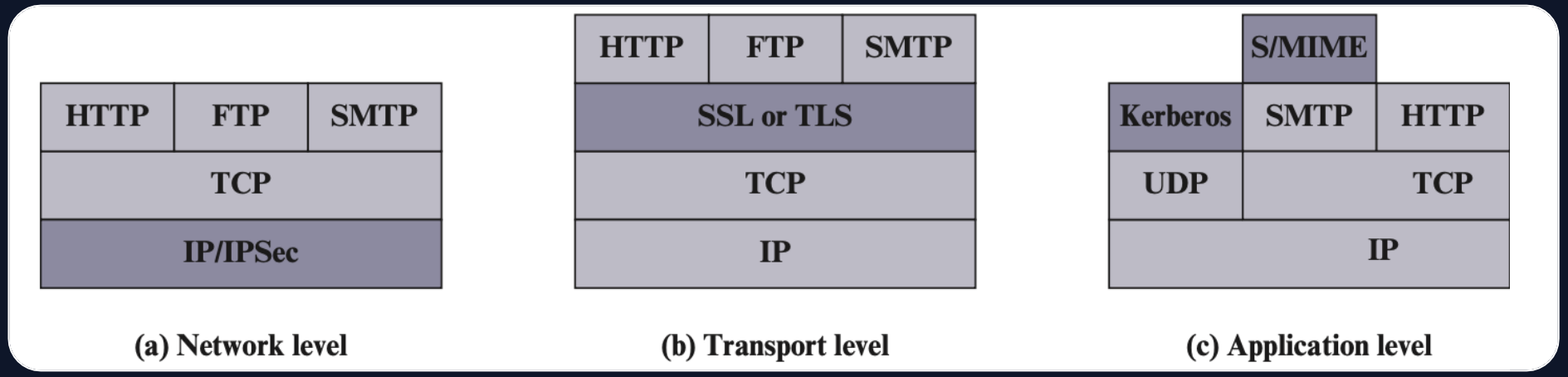


Image created with ChatGPT (OpenAI GPT-5.5)

© iSilva 2026

Securing end-to-end connections

- attacks target application entities (**client**, **server**, **peer processes**, etc.) or traffic between/among entities
  - passive: eavesdropping network traffic
  - active: impersonating user, altering data in transit (or in app/website)
- approaches to security in the protocol stack



[Stallings, 2023]

(a) general-purpose, transparent to L7, IP selective

(b) general purpose, embedded in app or not;

(c) app-tailored

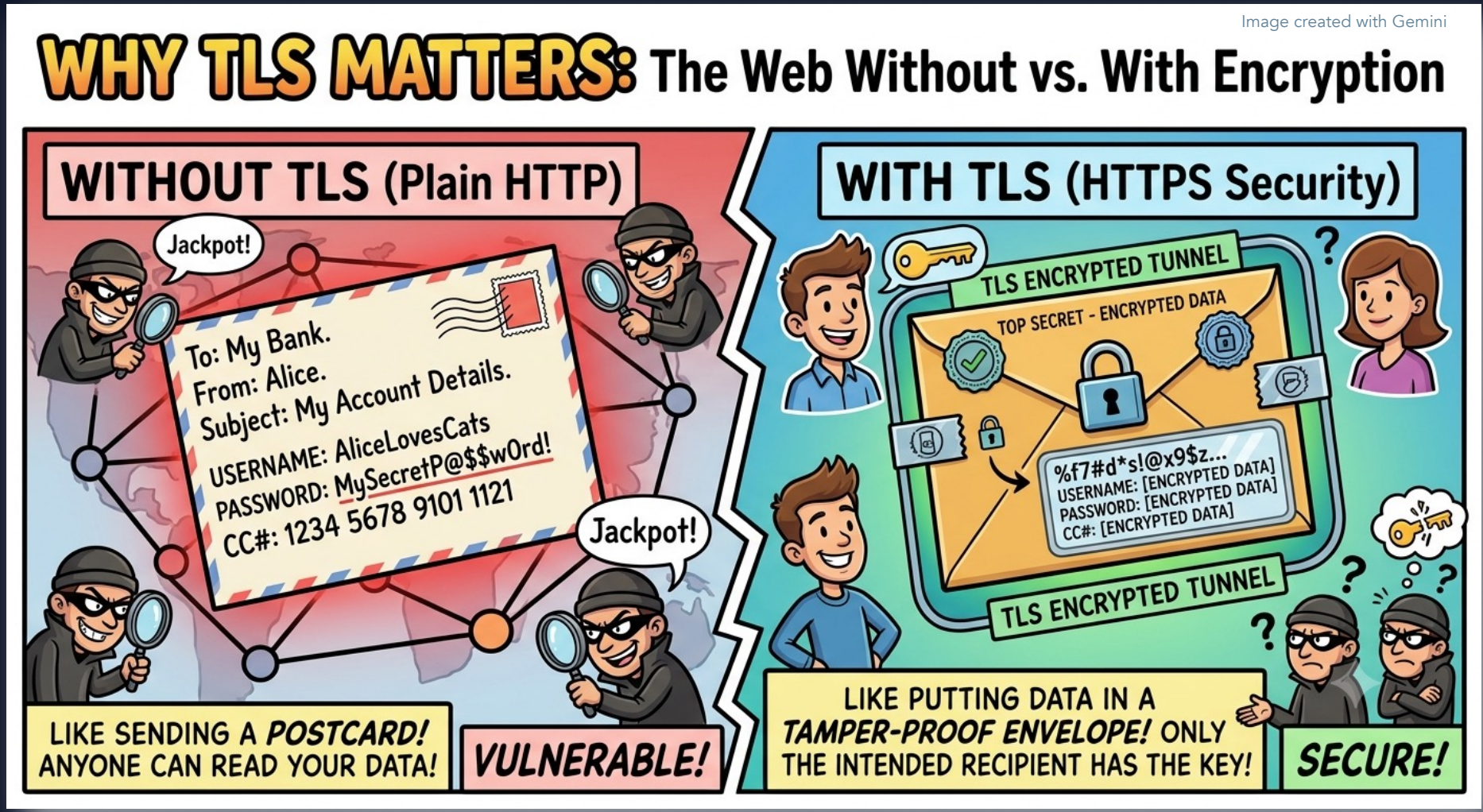
© iSilva 2026

Securing end-to-end connections

- **(a) Network Level**
  - Security is placed at the network layer, represented by IP/IPsec. This means that protection is more transparent to applications, because HTTP, FTP, or SMTP can run on top of it with little or no modification. However, the protection tends to be more general and selective, and it is not necessarily designed for the specific needs of each application.
- **(b) Transport Level**
  - Security appears at the transport layer, with SSL/TLS positioned between the application and TCP. This approach is also general purpose and very common, because it can protect communication between endpoints by providing confidentiality and integrity. However, it may be embedded into the application or used in a more generic way, depending on the protocol and the implementation.
- **(c) Application Level**
  - Security is implemented at the application layer, with examples such as S/MIME and Kerberos. In this model, protection is designed according to the needs of the application, so it is better adapted to the context. However, it becomes less transparent and usually requires specific logic in each service or protocol.

© iSilva 2026

Why Transport Layer Security (TLS) matters



© iSilva 2026

Transport-layer security (TLS)

def Transport-layer Security (TLS)

TLS is a widely used cryptographic protocol[+] designed to provide secure, end-to-end communications over a computer network by ensuring data privacy, integrity, and authentication.

- TLS primary goal is to provide a secure channel between communicating entities or peers
  - TLS primitives[*] can be accessed through various APIs that *any* application can use

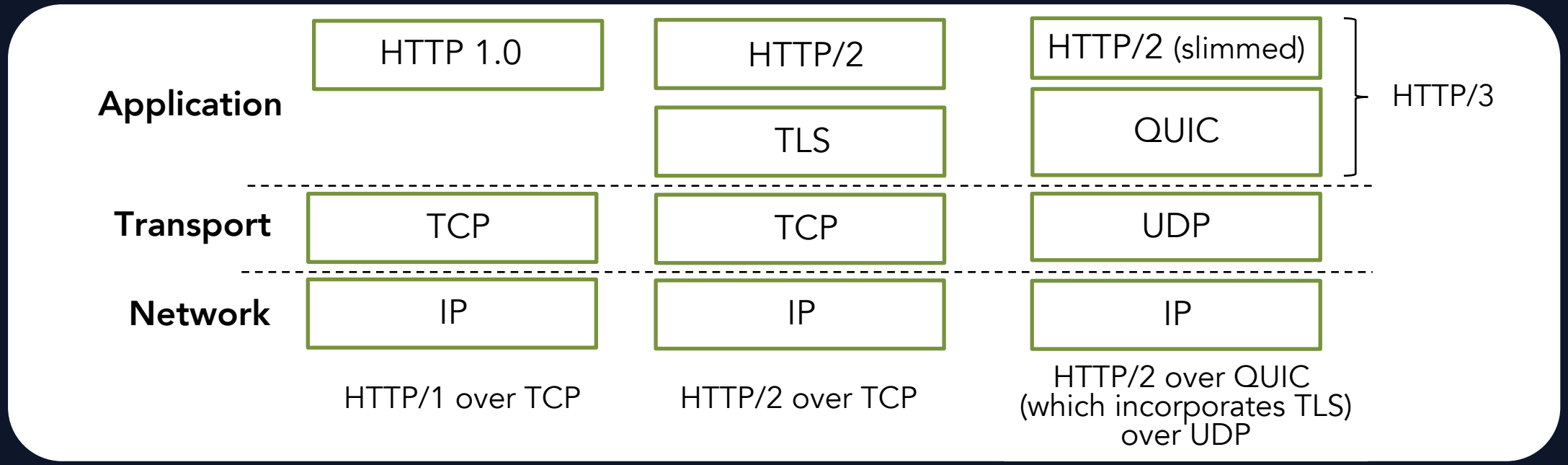
[+] **Cryptographic protocol**: is a set of rules that define how to use cryptographic primitives (encryption, authentication, hashing, key exchange, digital signatures) to provide security services like confidentiality, integrity, and authentication over communication channels.

[*] **TLS primitives**: TLS uses cryptographic primitives for encryption, authentication, hashing, key exchange, digital signatures.

© iSilva 2026

Transport-Layer Security (TLS)

- HTTP view of TLS:



- In **HTTP/1** and **HTTP/2**, secure communication is normally built by placing TLS on top of TCP.
- In **HTTP/3**, the transport changes from TCP to UDP, and security is handled inside QUIC, which already includes TLS related mechanisms. So the key idea is not only that TLS provides security, but also that protocol design has evolved and the stack is no longer exactly the same.

© iSilva 2026

Transport-Layer Security (TLS)

- widely deployed security protocol above the transport layer
  - supported by all recent browsers, web servers: https (port 443)
- provides:
  - confidentiality: via *symmetric encryption*
  - integrity: via *cryptographic hashing*
  - authentication: via *public key cryptography*

} *all techniques you have studied!*

- history:
  - early research, implementation (90's): secure net programming, secure sockets
  - Secure Socket Layer (SSL 3.0) deprecated [IETF, 2015]
  - TLS 1.3: RFC 8846 [2018] (TLS 1.2 RFC 5246 [2008] widely in use)

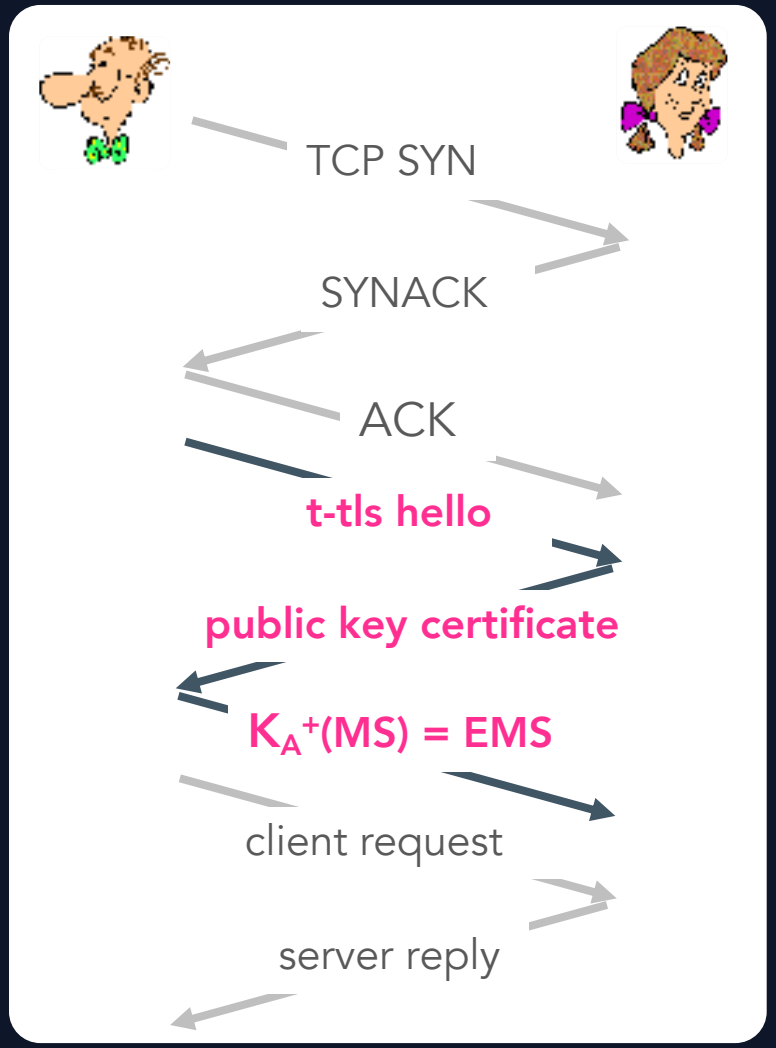
© iSilva 2026

Transport-layer security: what's needed?

- let's *build* a toy TLS protocol, *t-tls*, to see what's needed!
- required "pieces":
  - handshake: Alice, Bob use their certificates, private keys to authenticate each other, exchange or create shared secret
  - key derivation: Alice, Bob use shared secret to derive set of keys
  - data transfer: stream data transfer handled as a series of data records
    - not just one-time transaction
  - connection closure: special messages to securely close connection

© iSilva 2026

t-tls: initial handshake



t-tls handshake phase:

- Bob establishes TCP connection with Alice
- Bob then verifies Alice's identity using her public key certificate, making sure Alice **is really** Alice.
- After authentication, Bob sends Alice an encrypted master secret key (EMS) to Alice.
- Alice decrypts this value with her private key to recover the master secret using: K_A^- (EMS)
- The master secret is then used to derive all cryptographic keys for the TLS session.
- potential issues:
- 3 RTT before client can start receiving data (including TCP handshake)

© iSilva 2026

t-tls: cryptographic keys

- it's bad practice to use same key for more than one cryptographic function (important to separate contexts!)
  - different keys for Message Authentication Code (MAC)* and encryption
- four keys:
  - 🔑 K_c : encryption key for data sent from client to server
  - 🔑 M_c : MAC key for data sent from client to server (for integrity)
  - 🔑 K_s : encryption key for data sent from server to client
  - 🔑 M_s : MAC key for data sent from server to client
- keys are derived from a Key Derivation Function (KDF)
  - takes master secret and some additional (random) data to create new keys

**Message Authentication Code (MAC): is a cryptographic method used to verify the integrity and authenticity of a message. It is generated using a secret key and the message itself, creating a short "tag" that is sent with the message. The recipient uses the same secret key and MAC algorithm to generate their own tag and compares it to the received tag. A successful match confirms the message originated from the expected sender and has not been altered in transit.*

© iSilva 2026

t-tls: encrypting data

- **recall**: TCP provides data *byte stream* abstraction (i.e., application programs write bytes into the stream, and it is up to TCP to decide that it has enough bytes to send a segment)
- Q: can we encrypt data in-stream as written into TCP socket?
  - A: where would MAC go? If at end, no message integrity until all data received and connection closed!
  - solution: break data stream in series of "records"
    - each client-to-server record carries a MAC, created using M_c (MAC key)
    - receiver can act on each record as it arrives
- t-tls record encrypted using symmetric key, K_c, is passed to TCP:



© iSilva 2026

t-tls: encrypting data (more)

- **possible attacks** on data stream?
  - *re-ordering:* man-in-the-middle intercepts TCP segments and reorders (reordering, dropping, or injecting TLS records within the stream)
  - *Replay:* when an attacker intercepts legitimate data packets in a TCP session and resends them later to trick a server into repeating an action or granting unauthorized access.
- **solutions**:
  - use **local TLS sequence numbers** for sent and received records (data, implicit TLS-seq-# incorporated into MAC calculation)
    - e.g. `HMAC_hash(M_c, seq_num || type || length || data (plaintext))`
  - use **nonce** in symmetric cipher to guarantee unique encryption per record
  - changing the order or repeating a record changes the context and fails the verification

  (note: in TLS, as nonces can be derived from seq_# and IV (Initialization Vector), replay/reuse is prevented as long as seq_# never repeats under same IV/key)

© iSilva 2026

t-tls: connection close

- **truncation attack:**
  - attacker forges TCP connection close segment (FIN, RST)
  - one or both sides thinks there is less data than there actually is
- **solution:** use of record types, with one type for closure
  - type 0 for data; type 1 for close
- MAC now computed using length, type, sequence #, data
  - e.g. `HMAC_hash (Mc, length || type || seq_num || data (plaintext))`

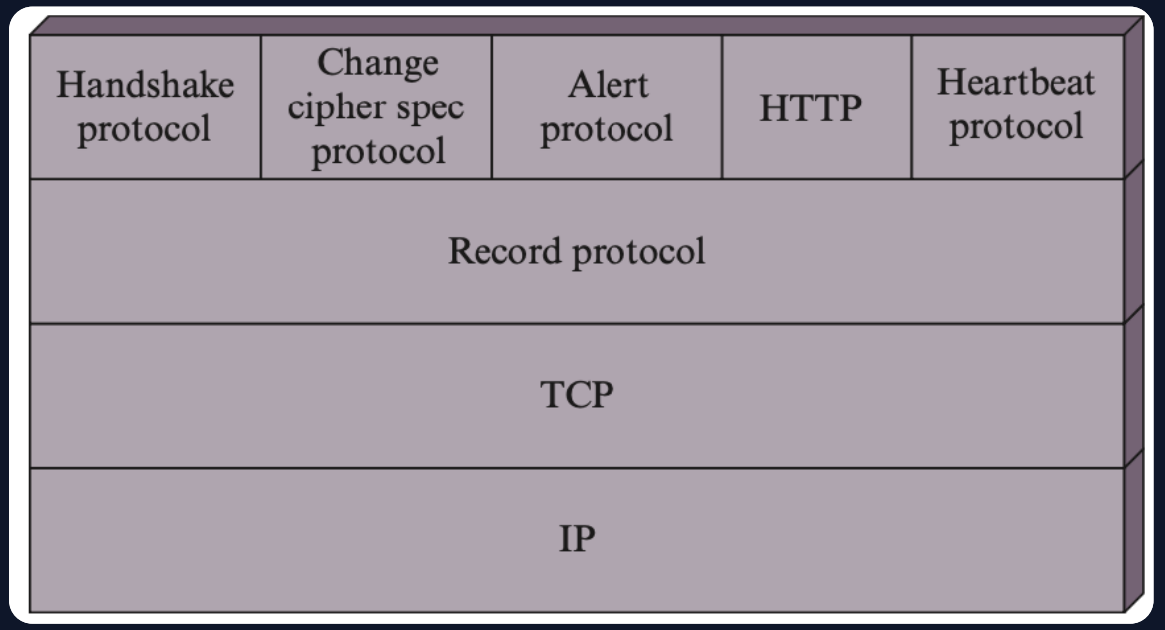


© iSilva 2026

Quizz

Transport Layer Security (TLS)

- TLS is a set of layered protocols
  - **Handshaking**
    - negotiate and establish a secure session
  - **Change cipher spec**
    - change to new encryption
  - **Alert**
    - report notification of problems
  - **Application**
    - send application-layer data, e.g. via HTTP
  - **Heartbeat**
    - keep alive session sensing
  - **Record**
    - package and secure application data



[Stallings, 2023]

© iSilva 2026

Transport Layer Security (TLS): concepts

def Session

A session is an association between the client and the server, which is created by TLS Handshake Protocol. It defines a set of cryptographic security parameters to be used by connections.

def Connection

A connection is an end-to-end temporary transport path between entities. Each connection is associated with a session.

© iSilva 2026

TLS Session – TLS 1.2 vs TLS 1.3

Session state TLS 1.2

- **Session ID** – ID set by server for the session (active, resumable)
- **Session Ticket** – encrypted unit with all session state sent to client
- **Peer certificate** – ITU-T X.509 v3 certificate (may be null)
- **Compression method** – to be used before encryption (mostly null)
- **Cipher spec** – defines the bulk data encryption algorithm (e.g. AES), a hash algorithm used for MAC calculation, other attributes (e.g. hash size)
- **Master Secret (MS)** – secret shared between parties (48 bytes)
  - derived locally from C-S randoms, Pre MS (PMS), and used in a KDF (Key Derivation Function) to obtain the session keys

Session state TLS 1.3

State management is simplified at client and server, namely:

- **Pre-Shared Key (PSK)** – instead of using session IDs or session tickets within the initial handshaking, PSK is introduced for fast session resumption
  - when a client establishes a connection, the server issues a session ticket that contains the encrypted PSK
  - PSK is used for session resumption without full handshaking
  - only the PSK and some session-related data (e.g., cipher suite) are stored by the client
  - minimal state is required by the server, typically just the PSK decryption key

© iSilva 2026

TLS Connection – TLS 1.2 vs TLS 1.3

Connection state TLS 1.2

- **C-S Randoms** – 32-bit nonces sent in Hello messages to use in KDF
- **MAC Keys** – for integrity checks (e.g., HMAC)
- **Session Keys** – r/w keys for the connection
- **Cipher Suite** – negotiated, for client and server
- **Initialization Vector (IV)** – maintained for each key
  - initiated during handshaking, then the last ciphertext block of each record is used as IV, for randomness
- **Sequence Numbers** – separate seq #s for sending/receiving (for data ordering integrity, and replay avoidance)
  - incremented monotonically per record sent, received
- **Finished Message** – to ensure that both parties agree on the negotiated session parameters (handshake validation)

Connection state TLS 1.3

- **Session Keys** – uses more secure key derivation and ephemeral key exchanges so that new set of keys is generated for every connection
- **Cipher Suite** – only supports strong, modern cipher suites (e.g., `TLS_AES_128_GCM_SHA256`, five in total)
- **Initialization Vector (IV)** – not negotiated, is generated internally as part of the Authenticated Encryption with Associated Data (AEAD) encryption process
- **Sequence Numbers** – each record uses seq #, always implicit, never transmitted
- **Finished Message** – to ensure that both parties agree on the negotiated session parameters (handshake validation)

© iSilva 2026

Transport Layer Security (TLS) – TLS 1.2 vs 1.3

Aspect	TLS 1.2	TLS 1.3
Init. Vector Management	IVs are explicitly negotiated or transmitted alongside encrypted data	IVs are derived from the handshake key and sequence number, handled internally to produce each record's nonce
Sequence Numbers	Separate read/write sequence numbers are maintained and implicitly managed	Separate read/write sequence numbers are maintained and implicitly managed
Encryption Algorithm	Both block ciphers (with CBC) and AEAD ciphers can be used	Only AEAD ciphers (e.g., AES-GCM, ChaCha20) are allowed (removed support for weak algorithms including MD5, SHA-1, RC4, DES, and 3DES)
Complexity	More complex due to separate management of IVs, sequence numbers, and cipher modes	Simplified with AEAD handling encryption, authentication, IVs, and sequence numbers

In summary:

- **TLS 1.3** simplifies the use of IVs and sequence numbers by incorporating them into the AEAD cipher process. The sequence #s are used to derive IVs internally, eliminating the need to send or explicitly manage them.
- **TLS 1.3** reduces the complexity of session and connection state and improves both security and performance

© iSilva 2026

From TLS 1.2 to TLS 1.3: Handshake Process and Performance

TLS 1.2

- involves a complex handshake with multiple round trips between Client and Server
- the full handshake requires 2-RTTs (client hello, server hello, key exchange)
- supports resuming a session through session IDs or session tickets
- clients and servers negotiate both the key exchange algorithm (e.g., Diffie-Hellman) and the encryption algorithm (e.g., AES, ChaCha20)

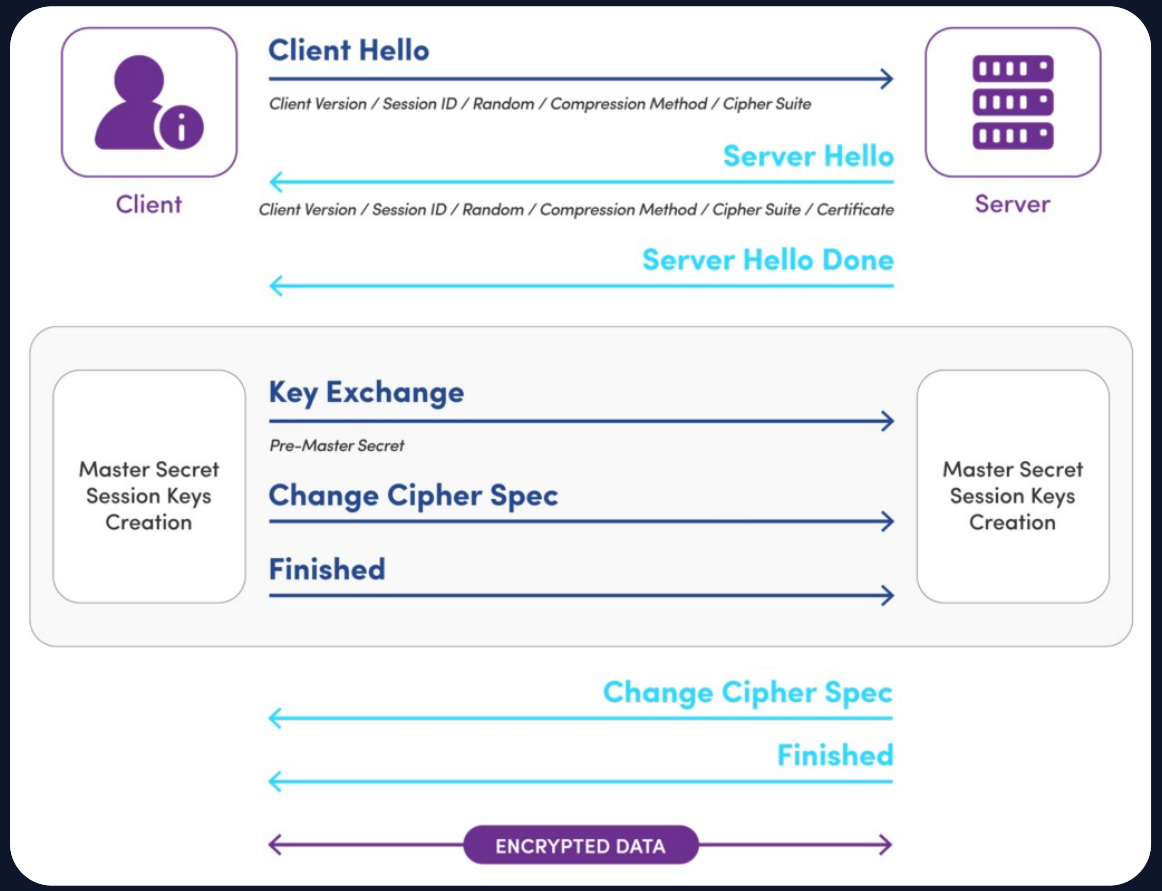
TLS 1.3

- simplifies the handshake process, reducing it to just 1-RTT for full handshakes
- when resuming a session, it can perform a "0-RTT" handshake, allowing encrypted data to be sent immediately after the client hello, drastically improving performance for repeated connections
- simplifies key negotiation, reducing the overhead and improving connection speeds

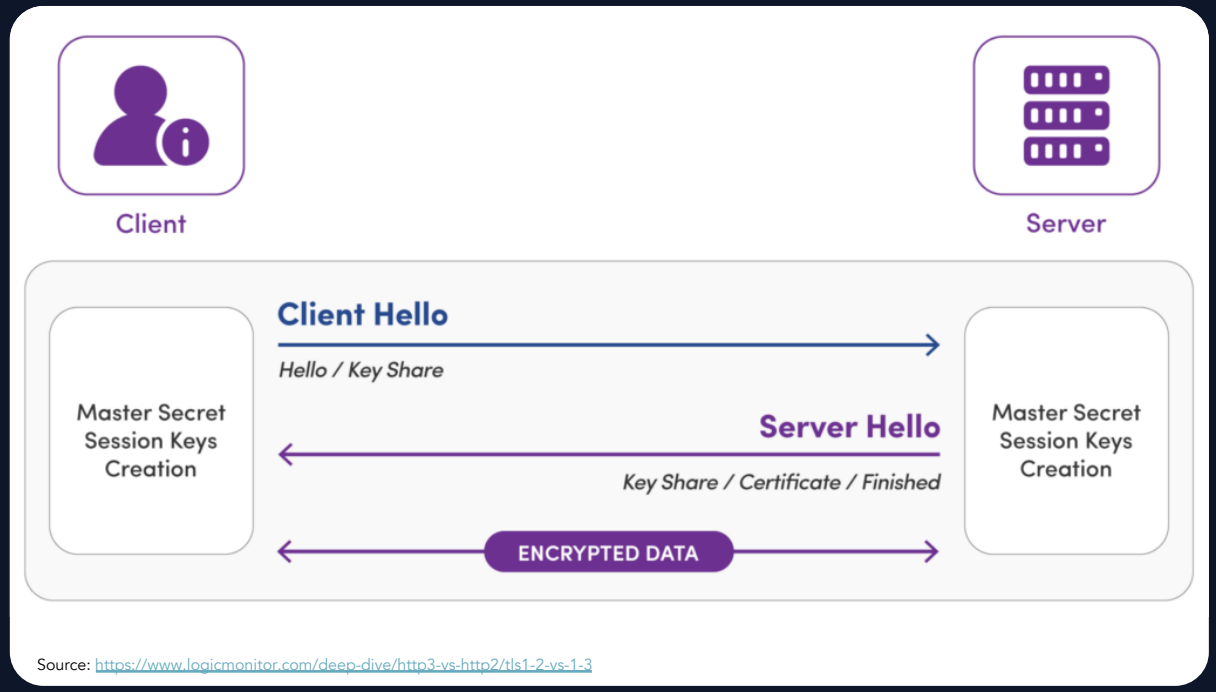
© iSilva 2026

Handshake Process: TLS 1.2 vs. TLS 1.3

TLS 1.2



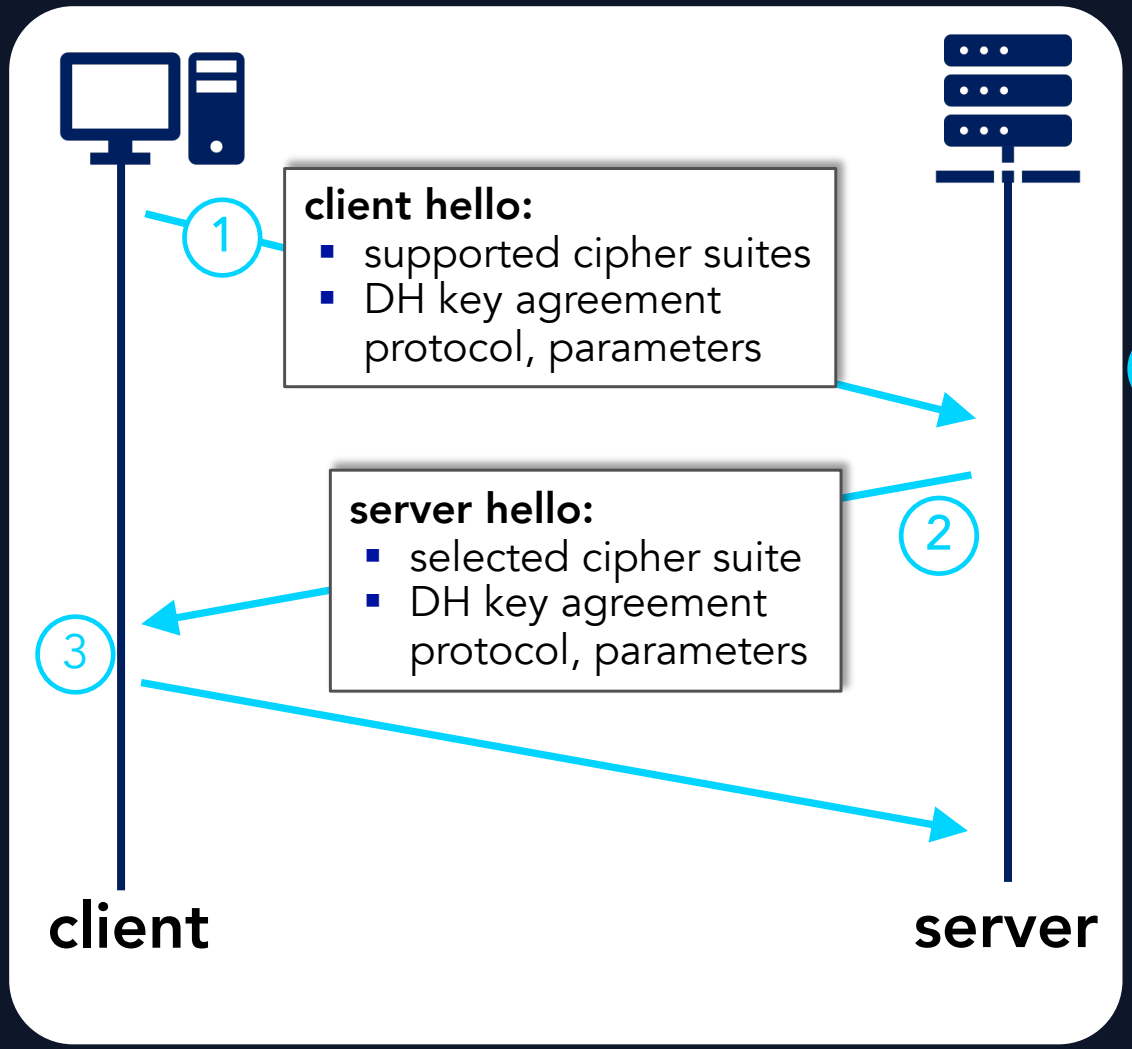
TLS 1.3



Source: https://www.logicmonitor.com/deep-dive/http3-vs-http2/tls1-2-vs-1-3

© iSilva 2026

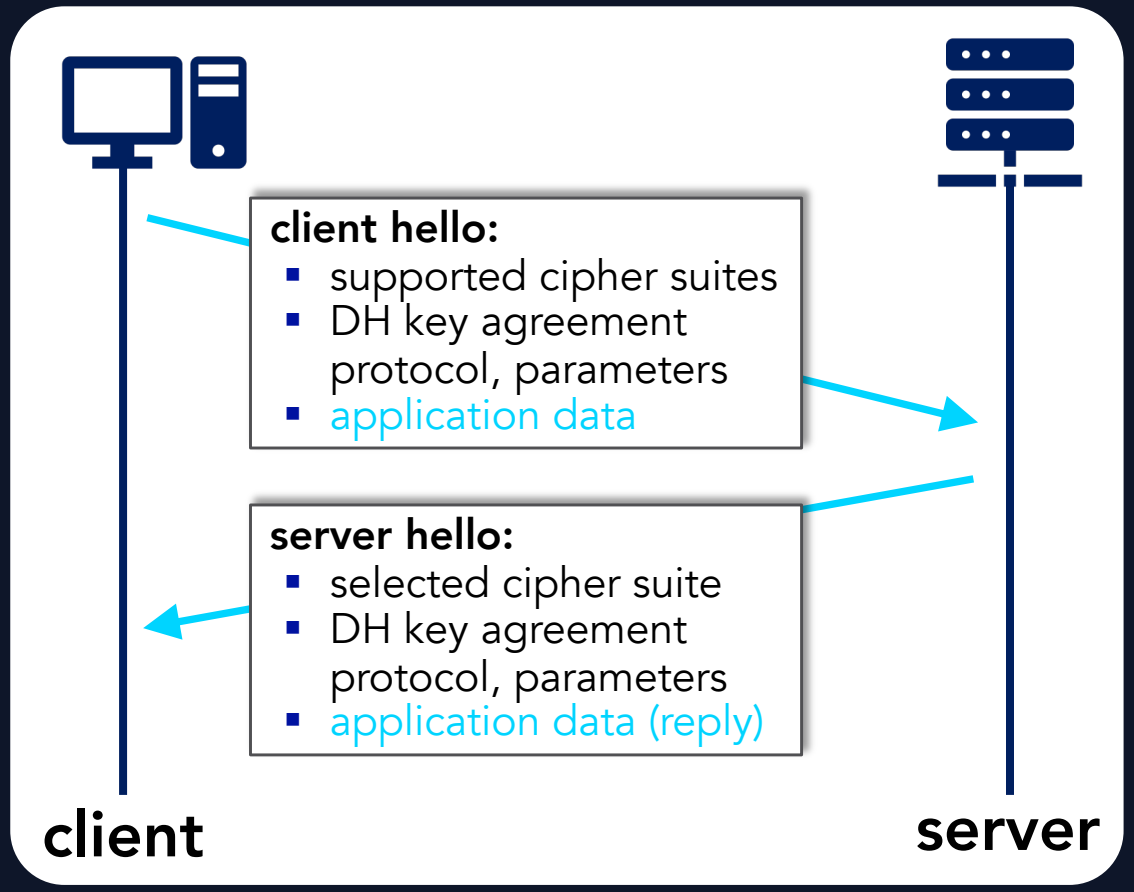
TLS 1.3 handshake: 1 RTT



1. client TLS hello msg:
   - proposes key agreement protocol, parameters
   - indicates cipher suites it supports
2. server TLS hello msg chooses
   - key agreement protocol, parameters
   - cipher suite
   - server-signed certificate
3. client:
   - checks server certificate
   - generates keys
   - can now make application request (e.g.., HTTPS GET)

© iSilva 2026

TLS 1.3 handshake: 0 RTT



- initial hello message contains encrypted application data!
  - "resuming" earlier connection between client and server
  - application data encrypted using "resumption master secret" from earlier connection (from master key (PSK) held in server´s cache)
- vulnerable to replay attacks*
  - maybe OK for HTTP GET or client requests not modifying server state
  - application logic must solve

Replay attack: When an attacker is able to capture the initial packet and replay it to trick a server into executing the same request.

© iSilva 2026

Transport Layer Security (TLS) – Cipher Suite

def Cipher Suite

Algorithms that can be used for key generation, encryption, MAC, digital signature

- TLS: 1.3 (2018): **simplified and more secure cipher suites than TLS 1.2** (2008)
  - only 5 choices, rather than 37 (less secure algorithms removed)
  - Removes RSA key transport; uses ephemeral (EC)DHE for certificate-based key exchange, *providing forward secrecy*
  - Requires authenticated encryption (AEAD), combining encryption and integrity protection.
  - 4 cipher suites based on AES (GCM/CCM) and 1 on ChaCha20-Poly1305.
  - Uses HKDF with SHA-256 or SHA-384 for cryptographic key derivation.
  - HMAC (Hash-based Message Authentication Code) combines a cryptographic hash function and a secret key; used within HKDF and for handshake authentication.

© iSilva 2026

From TLS 1.2 to TLS 1.3: Supported Cipher Suites

TLS 1.2

- supports a wide range of cryptographic algorithms, but not all are secure… e.g., weak algorithms such as RC4 and MD5, currently deprecated
- allows negotiating both the cipher suite and key exchange method separately, which can lead to security issues if insecure combinations are chosen
- uses block ciphers (e.g., AES in CBC mode) and stream ciphers (e.g., RC4)

TLS 1.3

- removed many older, insecure algorithms and cipher suites, including:
  - RSA key exchange; RC4, DES, 3DES, and other weak ciphers; MD5 and SHA-1 as hash functions.
- uses only secure, modern cryptographic algorithms, e.g. AES-GCM, ChaCha20-Poly1305
- combines key exchange and cipher suite negotiation into a single step, using AEAD (Authenticated Encryption with Associated Data) ciphers, which improve both security and performance

© iSilva 2026

From TLS 1.2 to TLS 1.3: Key Exchange Algorithms

TLS 1.2

- supports both RSA and Diffie-Hellman for key exchange
- RSA-based key exchange is less secure and vulnerable to certain types of attacks (e.g., forward secrecy is not guaranteed with RSA)
- Forward Secrecy is optional; therefore, past sessions data may be at risk if server private key is later compromised

TLS 1.3

- TLS 1.3 only supports Diffie-Hellman (DH) and Elliptic Curve Diffie-Hellman Ephemeral (ECDHE) key exchange, ensuring Perfect Forward Secrecy (PFS) in every session
- Forward Secrecy is always enforced, meaning even if long-term keys are compromised, past communication remains secure
- RSA key exchange has been completely removed

© iSilva 2026

From TLS 1.2 to TLS 1.3: Forward Secrecy

def Forward Secrecy

Forward secrecy (also known as Perfect Forward Secrecy or PFS) is a security feature that ensures a unique, temporary encryption key is generated for every single communication session. Even if an attacker compromises a server's long-term private key in the future, they cannot use it to retroactively decrypt past communications.

TLS 1.2

- Forward Secrecy (FS) is optional and depends on whether ephemeral key exchange protocol such as ECDHE is used
- Some cipher suites don't offer FS, making it possible for attackers to decrypt past communications if the private key was compromised

TLS 1.3

- Forward Secrecy is mandatory.
- All sessions use ephemeral key exchanges (such as ECDHE), ensuring that past communications cannot be decrypted even if the private key is compromised

© iSilva 2026

From TLS 1.2 to TLS 1.3: Privacy Improvements

TLS 1.2

- limited privacy enhancements, as parts of the handshake (such as the Server Name Indication or SNI) are sent in plaintext
- attackers may use the initial handshake information to gather data about the server's configuration (e.g., TLS version, cipher suites, server certificate and random)

TLS 1.3

- introduces Encrypted SNI (ESNI), which encrypts the Server Name Indication, to protect the destination server's identity during the handshake
- improves privacy by encrypting more of the handshake, including key exchange messages

© iSilva 2026

TLS 1.2 vs TLS 1.3: Comparison

Feature	TLS 1.2	TLS 1.3
Handshake	2-round trip full handshake	1-round trip full handshake, 0-RTT resumption
Key Exchange	RSA, DH, ECDHE (FS optional)	Only ECDHE (FS mandatory)
Encryption Algorithms	AES, RC4, DES, 3DES, etc.	Only secure options like AES-GCM, ChaCha20
Forward Secrecy	Optional, depends on key exchange	Always enforced
Cipher Suites	Wide variety, including weak ones	Simplified, only secure cipher suites
Session Resumption	Session IDs, tickets	PSK, 0-RTT resumption
Protocol Complexity	Complex, supports many legacy features	Simplified, removes insecure/legacy elements
Privacy Features	Minimal (e.g., SNI sent in plaintext)	Encrypted SNI, more handshake privacy
Vulnerabilities	Susceptible to downgrade and other attacks	Resists downgrade and protocol weaknesses

© iSilva 2026

Transport Layer Security: beyond TLS

- TLS is a unicast protocol

What if end-to-end is multiparty?

Current approaches (ongoing works with similar goals…)

- Group TLS (gTLS) – an extension of TLS for group communication (exp.)
- Multicast Security Protocols – GDOI; mTLS provide secure multicast comm.
  - Group Domain of Interpretation (GDOI) part of Multicast Security WG (MSEC)
  - Multicast TLS (mTLS) – extension of TLS
- Secure Group Messaging (SGM)
  - Messaging Layer Security (MLS) – IETF proposal for secure group messaging with support for large, dynamic groups
- Key Distribution and Management for Multiparty Communication
  - DH Group Key Exchange (DH-GKE); Tree-based Group Key Exchange (e.g., TreeKEM)

© iSilva 2026

Transport Layer Security: alternatives (1/2)

Current alternatives to TLS:

- **DTLS (Datagram Transport Layer Security)**
  - protocol based on TLS 1.3 (no order protection), designed for UDP
  - provides encryption and authentication for applications that use unreliable transport like VoIP or VPNs
  - RFC 9147 (Apr. 2022)
- **QUIC**
  - recent transport protocol, developed by Google
  - runs over UDP (to ease middlebox traversal)
  - incorporates many security features from TLS 1.3, offering built-in encryption and faster connection establishment
  - RFC 9000 (May 2021)

© iSilva 2026

Transport Layer Security: alternatives (2/2)

Signal Protocol: robust, open-source cryptographic system that provides end-to-end encryption for instant messaging, voice, and video calls. It is considered the gold standard for secure communication and is designed to protect conversations from being intercepted by third parties, including network providers and the server operators themselves.

- End-to-End Encryption:
  - while TLS focuses on securing transport channels (point-to-point), the Signal Protocol emphasizes end-to-end encryption (no intermediaries such as a third part proxy)
- Perfect Forward Secrecy and Deniability:
  - also present in TLS 1.3
  - adds deniability to allow participants to plausibly deny that they sent specific messages, which adds a layer of privacy (leave no traceable cryptographic evidence)
- Double Ratchet Mechanism
  - to continuously refresh encryption keys after every message, making it resilient to compromised keys over time
  - TLS uses a more static key agreement system (with occasional key renegotiation)
- Asynchronous Messaging
  - useful for modern messaging apps where participants may not be online simultaneously

© iSilva 2026

TLS: main limitations

- **Network inspection blind spots**: Encrypting the handshake makes it harder for corporate security tools to monitor traffic and block threats.
- **Replay attack risks (0-RTT)**: The ultra-fast connection feature (0-RTT) sends data immediately, allowing attackers to potentially intercept and repeat a request (like a payment) if the app is not properly designed.
- **No endpoint protection**: TLS only secures data in transit, meaning it cannot stop data or encryption keys from being stolen if the device or server itself is hacked.
- **Inability to stop malware or exploits**: The protocol creates a secure tunnel but does not inspect the payload, allowing viruses or web attacks (like SQL injection) to pass through freely.
- **TLS termination exposures**: Traffic is often decrypted at a load balancer or CDN before reaching the final backend server, leaving data exposed within the internal network.
- **Quantum computer vulnerability**: The current encryption methods used in TLS 1.3 will become vulnerable to being broken by future quantum computers.

© iSilva 2026

Quizz